Documentação de Requisitos

- Normas técnicas
- Estrutura padrão (modelos de documentação)
- Controle de Versões

Explicação do Tema

A documentação de requisitos é fundamental para garantir que todos os envolvidos em um projeto de software compreendam claramente o que deve ser desenvolvido. Os três elementos mencionados são pilares essenciais:

4.1. Normas Técnicas

As normas técnicas estabelecem padrões e diretrizes para a documentação de requisitos, garantindo qualidade, consistênc[6]ia e compreensão universal entre equipes. As principais incluem:

IEEE 830-1998 (Substituída pela IEEE 29148)

IEEE 830-1998 (Substituída pela IEEE 29148)

ISO/IEC/IEEE 29148:2018

ISO/IEC/IEEE 29148:2018

Framework CMMI (Capability Maturity Model Integration)

Framework CMMI (Capability Maturity Model Integration)

Exemplo Prático: Aplicação em README.md

Veja como documentar requisitos seguindo normas técnicas em um arquivo README.md:

None

```
# Sistema de Gestão de Tarefas - Especificação de Requisitos

## 1. Introdução

### 1.1 Propósito
Este documento especifica os requisitos funcionais e
não-funcionais para o Sistema de Gestão de Tarefas (SGT),
seguindo o padrão IEEE 29148.

### 1.2 Escopo
O SGT permitirá que usuários criem, organizem e acompanhem
tarefas pessoais e profissionais com sistema de prioridades e
prazos.

### 1.3 Definições e Acrônimos
- **SGT**: Sistema de Gestão de Tarefas
- **RF**: Requisito Funcional
- **RNF**: Requisito Não-Funcional
- **Sprint**: Período de 2 semanas de desenvolvimento

### 1.4 Referências
- IEEE 29148:2018 - Systems and software engineering
- CMMI for Development, Version 2.0

## 2. Descrição Geral

### 2.1 Perspectiva do Produto
O SGT será uma aplicação web responsiva com sincronização em
nuvem.

### 2.2 Funções Principais
- Criação e edição de tarefas
- Organização por projetos e tags
- Sistema de notificações
- Relatórios de produtividade

## 3. Requisitos Específicos

### 3.1 Requisitos Funcionais
```

RF-001: Criação de Tarefas
Descrição: O sistema deve permitir que usuários criem tarefas com título, descrição, data de vencimento e prioridade.

Prioridade: Alta
Versão: 1.0
Data: 2026-03-25
Rastreabilidade: Derivado de necessidade do stakeholder NS-001

Critérios de Aceitação:
- [ ] Formulário com campos obrigatórios (título) e opcionais
- [ ] Validação de data (não permitir datas passadas)
- [ ] Níveis de prioridade: Baixa, Média, Alta, Urgente
- [ ] Confirmação visual após criação

Dependências: Nenhuma

RF-002: Organização por Projetos
Descrição: O sistema deve permitir agrupar tarefas em projetos personalizados.

Prioridade: Média
Versão: 1.0
Data: 2026-03-25
Rastreabilidade: Derivado de NS-002

Critérios de Aceitação:
- [ ] Usuário pode criar, renomear e excluir projetos
- [ ] Tarefas podem ser atribuídas a um ou nenhum projeto
- [ ] Visualização filtrada por projeto

Dependências: RF-001

3.2 Requisitos Não-Funcionais

RNF-001: Desempenho
Descrição: O sistema deve carregar a lista de tarefas em menos de 1 segundo para até 1000 tarefas.

Categoria: Desempenho
Prioridade: Alta
Versão: 1.0
Métrica: Tempo de resposta < 1s para 95% das requisições

RNF-002: Segurança
Descrição: O sistema deve implementar autenticação OAuth 2.0 e criptografia TLS 1.3.

Categoria: Segurança
Prioridade: Crítica
Versão: 1.0
Conformidade: LGPD, GDPR

---

4. Controle de Versões

Histórico de Alterações

Versão	Data	Autor	Modificações
1.0	2026-03-25	Equipe Análise	Versão inicial do documento
1.1	2026-03-30	João Silva	Adicionado RF-003 (notificações)
1.2	2026-04-05	Maria Santos	Atualizado RNF-001 (métrica revisada)

```
### Rastreabilidade

```mermaid
graph LR
    NS001["NS-001: Necessidade Stakeholder"] -->
RF001["RF-001: Criação Tarefas"]
    NS002["NS-002: Organização"] --> RF002["RF-002: Projetos"]
    RF001 --> TC001["TC-001: Teste Criação"]
    RF002 --> TC002["TC-002: Teste Projetos"]
```

5. Aprovações

Papel	Nome	Data	Assinatura
Product Owner	Ana Costa	2026-03-25	✓
Arquiteto Software	Carlos Lima	2026-03-25	✓
Líder QA	Beatriz Souza	2026-03-26	✓

6. Apêndices

Apêndice A: Glossário

- **Tarefa**: Unidade básica de trabalho no sistema
- **Projeto**: Agrupamento lógico de tarefas relacionadas
- **Prioridade**: Nível de importância atribuído a uma tarefa

Apêndice B: Modelos de Análise

- Diagrama de Casos de Uso
- Diagrama de Classes
- Fluxogramas de processos principais
```

None

Benefícios desta Abordagem

- **Rastreabilidade:** Cada requisito tem ID único e referências claras
- **Versionamento:** Histórico completo de alterações documentado
- **Conformidade:** Segue padrões internacionais (IEEE 29148)
- **Clareza:** Critérios de aceitação específicos e mensuráveis
- **Manutenibilidade:** Estrutura organizada facilita atualizações
- **Comunicação:** Formato Markdown facilita colaboração em Git/GitHub

Essas normas garantem consistência, completude e rastreabilidade dos requisitos ao longo de todo o ciclo de vida do projeto.

4.2. Estrutura Padrão (Modelos de Documentação)

Um documento de requisitos bem estruturado geralmente contém:

1. **Introdução:** Propósito, escopo, definições e referências
2. **Descrição Geral:** Perspectiva do produto, funções principais, características dos usuários
3. **Requisitos Específicos:** Requisitos funcionais e não-funcionais detalhados
4. **Apêndices:** Informações complementares, glossário, modelos de análise

Modelos comuns incluem: Casos de Uso, User Stories (Histórias de Usuário), Especificação Formal, e Protótipos.

4.2.1. Introdução

A seção de introdução estabelece o contexto do projeto:

```
- **Propósito:** Define claramente por que o documento existe
e quais problemas o sistema resolverá
- **Escopo:** Delimita o que está incluído e, igualmente
importante, o que está fora do escopo
- **Definições e Acrônimos:** Garante vocabulário comum entre
todas as partes interessadas
- **Referências:** Lista documentos relacionados, normas
aplicadas e fontes de informação

**Exemplo prático:**

```markdown
## 1. Introdução

### 1.1 Propósito
Este documento especifica os requisitos para o Sistema de
Controle Acadêmico (SCA),
que digitalizará processos de matrícula, notas e frequência da
universidade.

### 1.2 Escopo
**Incluído:** Gestão de matrículas, lançamento de notas,
controle de frequência
**Excluído:** Sistema financeiro, biblioteca, recursos humanos

### 1.3 Definições
- **Discente:** Estudante regularmente matriculado
- **Docente:** Professor vinculado à instituição
- **CR:** Coeficiente de Rendimento

### 1.4 Referências
- IEEE 29148:2018
- Regimento Acadêmico da Universidade (2025)
```

4.2.2. Descrição Geral

Fornece visão de alto nível do sistema:
```

- **Perspectiva do Produto:** Como o sistema se relaciona com outros sistemas e ambiente operacional
- **Funções Principais:** Principais capacidades sem detalhamento técnico
- **Características dos Usuários:** Perfis, habilidades técnicas e frequência de uso
- **Restrições:** Limitações tecnológicas, regulatórias ou de negócio

Exemplo prático:

```
None
## 2. Descrição Geral

### 2.1 Perspectiva do Produto
O SCA será integrado ao sistema de autenticação existente
(SSO) e consumirá
dados do sistema de RH para informações de professores.

### 2.2 Funções Principais
- Matrícula online em disciplinas
- Registro eletrônico de frequência
- Lançamento e consulta de notas
- Geração de histórico acadêmico

### 2.3 Características dos Usuários
| Tipo | Frequência | Habilidade Técnica | Acesso |
|------|------------|-------------------|--------|
| Aluno | Diária (período de matrícula) / Semanal (consultas)
| Básica | Web/Mobile |
| Professor | Semanal (frequência) / Mensal (notas) |
Intermediária | Web |
| Secretaria | Diária | Avançada | Web (admin) |

### 2.4 Restrições
- Conformidade com LGPD para dados pessoais de estudantes
- Disponibilidade 99.5% durante período letivo
- Compatibilidade com navegadores dos últimos 2 anos
```

4.2.3. Modelos de Documentação

A) Casos de Uso

Descrevem interações entre atores e sistema:

```None
### Caso de Uso: UC-001 - Realizar Matrícula

**Ator Principal:** Aluno
**Pré-condições:** Aluno autenticado, período de matrícula aberto
**Fluxo Principal:**
1. Aluno acessa menu "Matrícula"
2. Sistema exibe disciplinas disponíveis com horários e vagas
3. Aluno seleciona disciplinas desejadas
4. Sistema valida pré-requisitos e conflitos de horário
5. Aluno confirma matrícula
6. Sistema registra matrícula e envia confirmação por email

**Fluxos Alternativos:**
- 4a. Se houver conflito de horário: Sistema exibe mensagem de erro e retorna ao passo 3
- 4b. Se faltar pré-requisito: Sistema lista disciplinas pendentes e retorna ao passo 3

**Pós-condições:** Matrícula registrada no sistema
```

B) User Stories (Histórias de Usuário)

Formato ágil focado no valor para o usuário:

```None
### US-001: Visualizar Notas

**Como** aluno
**Quero** visualizar minhas notas por disciplina
**Para que** eu possa acompanhar meu desempenho acadêmico

**Critérios de Aceitação:**
- [ ] Exibir notas de todas as avaliações (provas, trabalhos, seminários)
- [ ] Mostrar média parcial e final
- [ ] Indicar visualmente se está aprovado (média ≥ 7.0)
- [ ] Permitir exportar boletim em PDF
```

```
- [ ] Atualização em tempo real quando professor lançar notas

**Estimativa:** 5 pontos
**Prioridade:** Alta
**Sprint:** 2
```

C) Especificação Formal

Para requisitos críticos que exigem precisão matemática:

```
None
### RF-015: Cálculo de Média Final

**Especificação Formal:**

Seja:
- P1, P2, P3 = notas das provas (0 ≤ Pi ≤ 10)
- T = nota dos trabalhos (0 ≤ T ≤ 10)
- F = frequência percentual (0 ≤ F ≤ 100)

Média Final (MF):
MF = (P1 × 0.25) + (P2 × 0.25) + (P3 × 0.30) + (T × 0.20)

Condições de Aprovação:
- Aprovado: MF ≥ 7.0 E F ≥ 75%
- Reprovado por Nota: MF < 5.0
- Reprovado por Falta: F < 75%
- Recuperação: 5.0 ≤ MF < 7.0 E F ≥ 75%
```

Exercício Prático: Sistema de Avaliação de Cursos

Objetivo: Criar documentação de requisitos usando Git/GitHub com controle de versões colaborativo

Cenário

Sua equipe foi contratada para desenvolver um sistema onde alunos avaliam disciplinas e professores ao final do semestre. A coordenação usará esses dados para melhorias acadêmicas.

Etapa 1: Configuração do Repositório (Individual)

```
None
# 1. Criar repositório no GitHub
# Nome: sistema-avaliacao-cursos
# Incluir: README.md, .gitignore (templates para Python ou Node)

# 2. Clonar localmente
git clone
https://github.com/seu-usuario/sistema-avaliacao-cursos.git
cd sistema-avaliacao-cursos

# 3. Criar estrutura de pastas
mkdir docs
touch docs/requisitos.md
touch docs/casos-de-uso.md
touch docs/user-stories.md

# 4. Primeiro commit
git add .
git commit -m "docs: estrutura inicial do projeto"
git push origin main
```

Etapa 2: Documentação de Requisitos (Equipes de 3-4 alunos)

Tarefa A - Requisitos Funcionais (Aluno 1):

```
None
# Criar branch para trabalhar
git checkout -b feat/requisitos-funcionais

# Editar docs/requisitos.md com pelo menos 8 requisitos
funcionais
```

```
# Seguir padrão IEEE 29148
```

Conteúdo mínimo do arquivo:

```
None
# Requisitos Funcionais

## RF-001: Autenticação de Usuários
**Descrição:** O sistema deve autenticar alunos via
credenciais institucionais (SSO)
**Prioridade:** Crítica
**Versão:** 1.0
**Data:** 2026-03-25
**Autor:** [Seu Nome]

## RF-002: Seleção de Disciplina para Avaliação
**Descrição:** O sistema deve exibir apenas disciplinas
cursadas pelo aluno no semestre atual
**Prioridade:** Alta
**Versão:** 1.0

[... continuar com RF-003 a RF-008]
```

```
None
# Commit e push
git add docs/requisitos.md
git commit -m "docs(requisitos): adiciona requisitos
funcionais RF-001 a RF-008"
git push origin feat/requisitos-funcionais

# Abrir Pull Request no GitHub
```

Tarefa B - Casos de Uso (Aluno 2):

```
None
git checkout -b feat/casos-de-uso

# Criar pelo menos 3 casos de uso detalhados em
docs/casos-de-uso.md
```

Exemplo de caso de uso:

```
None
# Casos de Uso

## UC-001: Avaliar Disciplina

**Ator:** Aluno
**Pré-condições:**
- Aluno autenticado
- Período de avaliação aberto
- Aluno cursou a disciplina no semestre

**Fluxo Principal:**
1. Aluno acessa "Minhas Avaliações"
2. Sistema lista disciplinas cursadas não avaliadas
3. Aluno seleciona disciplina
4. Sistema exibe formulário com:
   - Notas de 1-5 para critérios (didática, clareza,
pontualidade, etc.)
   - Campo texto para comentários (opcional)
   - Checkbox "Recomendo esta disciplina"
5. Aluno preenche formulário
6. Sistema valida preenchimento obrigatório
7. Aluno confirma envio
8. Sistema registra avaliação anonimizada
9. Sistema exibe mensagem de sucesso

**Fluxos Alternativos:**
- 6a. Campos obrigatórios vazios: Exibir mensagem e destacar
campos
```

```
**Pós-condições:** Avaliação armazenada, disciplina marcada
como avaliada
```

Tarefa C - User Stories (Aluno 3):

```
None
git checkout -b feat/user-stories

# Criar 10 user stories em docs/user-stories.md
```

Etapa 3: Controle de Versões e Evolução

Simulação de Mudança de Requisitos:

```
None
# Após merge de todas as branches, o professor solicita
alteração:
# "O sistema deve permitir avaliações por QR Code nas salas de
aula"

# Aluno 1 cria branch
git checkout main
git pull origin main
git checkout -b feat/atualizacao-requisitos-v1.1

# Atualizar requisitos.md
```

Adicionar ao arquivo:

```
None
## Histórico de Versões

| Versão | Data       | Autor          | Modificações
|
```

```
|--------|------------|----------------|----------------------
----------------------|
| 1.0    | 2026-03-25 | Equipe 1       | Versão inicial -
RF-001 a RF-008           |
| 1.1    | 2026-03-28 | João Silva     | Adicionado RF-009
(avaliação via QR Code)  |

## RF-009: Avaliação via QR Code
**Descrição:** O sistema deve gerar QR Code único por
disciplina que redireciona para formulário de avaliação
**Prioridade:** Média
**Versão:** 1.1
**Data:** 2026-03-28
**Rastreabilidade:** Solicitação do stakeholder em reunião
28/03/2026
```

Etapa 4: Revisão Colaborativa (Code Review)

Cada aluno deve:

- Revisar pelo menos 2 Pull Requests de colegas
- Deixar comentários construtivos sobre clareza, completude e aderência às normas
- Aprovar ou solicitar alterações

Exemplo de comentário em PR:

```
None
**Revisão do RF-003:**

✅ Pontos positivos:
- Descrição clara e objetiva
- Prioridade bem justificada

⚠️ Sugestões:
- Adicionar critérios de aceitação específicos
- Incluir rastreabilidade (de qual necessidade do stakeholder
veio)
```

```
- Especificar comportamento quando não há disciplinas
disponíveis
```

Etapa 5: Entrega Final

Estrutura esperada do repositório:

```
None
sistema-avaliacao-cursos/
├── README.md (visão geral do projeto)
├── docs/
│   ├── requisitos.md (com versionamento)
│   ├── casos-de-uso.md
│   ├── user-stories.md
│   └── rastreabilidade.md (matriz requisitos ↔ casos de uso ↔
user stories)
├── .gitignore
└── CHANGELOG.md (histórico de todas as versões)
```

Critérios de Avaliação

Critério	Peso	Descrição
Qualidade dos Requisitos	30%	Clareza, completude, aderência à IEEE 29148
Uso do Git	25%	Commits atômicos, mensagens descritivas, branches organizadas
Versionamento	20%	Histórico documentado, rastreabilidade implementada
Colaboração (PRs)	15%	Reviews construtivos, resolução de conflitos
Documentação Geral	10%	README completo, CHANGELOG atualizado

Bônus: Automação com GitHub Actions

Criar arquivo `.github/workflows/docs-validation.yml`:

```
None
name: Validação de Documentação

on: [pull_request]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Verificar formatação Markdown
        run: |
          npm install -g markdownlint-cli
          markdownlint docs/**/*.md

      - name: Verificar padrão de requisitos
        run: |
          # Script para validar que todos os requisitos têm
ID, Descrição, Prioridade, Versão
          python scripts/validate_requirements.py
```

Este exercício ensina simultaneamente:

- Documentação técnica padronizada
- Fluxo de trabalho Git/GitHub profissional
- Colaboração em equipe
- Versionamento e rastreabilidade de requisitos
- Code review e qualidade

4.3. Controle de Versões

O controle de versões é essencial para rastrear mudanças nos requisitos ao longo do tempo:

- **Versionamento semântico:** Uso de números (ex: v1.0, v1.1, v2.0) para identificar versões
- **Histórico de alterações:** Registro detalhado de quem modificou, quando e por quê

- **Rastreabilidade:** Ligação entre requisitos e suas versões anteriores
- **Ferramentas:** Git, SVN, sistemas de gerenciamento de requisitos (JIRA, Azure DevOps)

Vou expandir o conteúdo sobre Controle de Versões com mais detalhes e exemplos práticos.

Por que o Controle de Versões é Crítico?

Durante o ciclo de vida de um projeto, requisitos sofrem alterações por diversos motivos:

- Mudanças nas necessidades do cliente
- Descoberta de requisitos ocultos durante o desenvolvimento
- Alterações regulatórias ou de mercado
- Feedback de usuários em protótipos
- Restrições técnicas ou orçamentárias identificadas tardiamente

Sem um controle adequado, essas mudanças podem causar:

- Perda de rastreabilidade (não saber por que uma decisão foi tomada)
- Conflitos entre equipes trabalhando com versões diferentes
- Retrabalho por falta de sincronização
- Dificuldade em auditorias e certificações

Elementos de um Bom Sistema de Versionamento

1. Versionamento Semântico Adaptado

Para requisitos, uma abordagem comum é:

- **Major.Minor.Patch** (ex: 2.3.1)
    - **Major:** Mudanças estruturais significativas (ex: novo módulo do sistema)
    - **Minor:** Adição ou modificação de requisitos existentes
    - **Patch:** Correções de erros, clarificações, ajustes de texto

2. Matriz de Rastreabilidade

Tabela que conecta:

- Requisitos ↔ Stakeholders que solicitaram
- Requisitos ↔ Casos de uso que implementam
- Requisitos ↔ Casos de teste que validam
- Requisitos ↔ Código fonte (módulos/classes)

3. Baseline (Linha de Base)

Snapshot aprovado formalmente dos requisitos em um momento específico. Serve como referência para medir mudanças futuras.

Exemplos Detalhados

Exemplo 1: Evolução de um Requisito em Sistema Bancário

Versão 1.0 (Janeiro 2026)

[redacted]

Versão 1.1 (Fevereiro 2026)

[redacted]

Versão 2.0 (Março 2026)

[redacted]

Exemplo 2: Controle de Versões com Git

Estrutura de commits para documentação de requisitos:

```
None
# Commit inicial
git commit -m "docs(req): RF-042 v1.0 - transferência entre
contas internas"

# Adição de funcionalidade
git commit -m "docs(req): RF-042 v1.1 - adiciona suporte
TED/PIX
BREAKING CHANGE: Requer integração com API PIX do Banco
Central
Refs: #234 (issue do JIRA)"

# Mudança regulatória
git commit -m "docs(req): RF-042 v2.0 - limites PIX
configuráveis
Motivo: Resolução BCB 4.999/2026
Impacto: 3 sprints de desenvolvimento adicional
Stakeholder: Compliance + Gerência Comercial"
```

Exemplo 3: Matriz de Rastreabilidade

Requisito	Versão	Caso de Uso	User Story	Teste	Status
RF-042	2.0	UC-015, UC-016	US-087, US-088	TC-234 a TC-241	Em desenvolvimento
RF-043	1.3	UC-017	US-089	TC-242 a TC-245	Testado
RF-044	1.0	UC-018	US-090	TC-246 a TC-248	Aprovado

Exercício Prático: Sistema de Controle de Versões de Requisitos

Cenário: Você é analista de requisitos de um sistema de reservas de hotéis. Um requisito inicial foi modificado duas vezes ao longo do projeto.

Requisito Original (Janeiro 2026):

[redacted]

Mudança 1 (Fevereiro 2026):

O departamento de vendas solicitou: "Clientes VIP devem poder reservar com até 1 ano de antecedência."

Mudança 2 (Março 2026):

Nova política da empresa: "Reservas de última hora (menos de 2 horas de antecedência) agora são permitidas, mas com taxa adicional de 15%."

Tarefas:

1. **Versionamento:** Crie três versões documentadas do requisito (RF-010) seguindo o padrão apresentado nos exemplos, incluindo:
    - ID do requisito
    - Descrição completa
    - Número de versão (use versionamento semântico)
    - Data de cada versão
    - Autor responsável pela mudança

    - Motivo da alteração
    - Impacto técnico ou de negócio
2. **Matriz de Rastreabilidade:** Crie uma tabela conectando:
    - RF-010 (versões 1.0, 2.0, 3.0)
    - Casos de uso relacionados (invente IDs: UC-XXX)
    - User stories (invente IDs: US-XXX)
    - Testes necessários (invente IDs: TC-XXX)
3. **Mensagens de Commit:** Escreva as mensagens de commit Git que você usaria para registrar cada versão do requisito no repositório.
4. **Análise de Impacto:** Para a versão 3.0, identifique:
    - Quais outros requisitos podem ser afetados (ex: pagamentos, notificações)
    - Quais stakeholders devem aprovar a mudança
    - Estimativa de esforço de desenvolvimento (baixo/médio/alto)

Formato de Entrega Esperado:

Um documento Markdown estruturado contendo:

- Seção "Histórico de Versões do RF-010" com as três versões
- Seção "Matriz de Rastreabilidade"
- Seção "Registro Git"
- Seção "Análise de Impacto da v3.0"

Critérios de Avaliação:

- Completude das informações de versionamento (20 pontos)
- Clareza na descrição das mudanças e motivos (25 pontos)
- Qualidade da matriz de rastreabilidade (20 pontos)
- Mensagens de commit seguindo boas práticas (15 pontos)
- Análise de impacto realista e detalhada (20 pontos)

Este exercício simula situações reais de projetos ágeis onde requisitos evoluem constantemente e a rastreabilidade é fundamental para o sucesso.